

This repository was archived by the owner on Jun 30, 2026. It is now read-only.

StefanTerdell / zod-to-json-schema Public archive

[Code](#) [Issues](#) [Pull requests](#) [Actions](#) [Security and quality](#) [Insights](#)

9 Branches

7 Tags



Go to file

Go to file

Code



StefanTerdell



3.25.2

a001370 · 5 months ago

.github	Add sponsor logo	last year
dist-test-v3	3.25.2	5 months ago
dist-test-v4	3.25.2	5 months ago
src	Remove c8 mentions	9 months ago
test	chore: allow zod 4 peer	last year
.gitignore	Fix #149 - support branded key schemas	2 years ago
.npmignore	Update .npmignore to include dist-test pattern	8 months ago
.prettierrc.json	Format project	3 years ago
LICENSE	Add author and license	5 years ago
README.md	Update deprecation notice	5 months ago
changelog.md	3.25.2	5 months ago
contributing.md	Build stuff (esm)	3 years ago
createIndex.ts	Fix index generators	3 years ago
package-lock.js...	3.25.2	5 months ago
package.json	3.25.2	5 months ago
postcjs.ts	Postscripts js -> ts	2 years ago
postesm.ts	Audit fixes	last year

 tsconfig.cjs.json	 Build stuff (esm)	3 years ago
 tsconfig.esm.json	 Format project	3 years ago
 tsconfig.json	 Add test for #162	last year
 tsconfig.types.j...	 Build stuff (esm)	3 years ago

README	Contributing	ISC license	
--------	--------------	-------------	---

Zod to Json Schema

npm **v3.25.2** downloads **58M/week**

Notice of deprecation

As of November 2025, this project will no longer be actively maintained. [Zod v4 natively supports generating JSON schemas](#), so I recommend you switch to the new major.

Thank you to all the contributors and sponsors throughout the years! So long, and thanks for all the fish.

Summary

Does what it says on the tin; converts [Zod schemas](#) into [JSON schemas](#)!

- Supports all relevant schema types, basic string, number and array length validations and string patterns.
- Resolves recursive and recurring schemas with internal `$ref`s.
- Supports targeting legacy Open API 3.0 specification (3.1 supports regular Json Schema).
- Supports Open AI strict mode schemas (Optional object properties are replaced with required but nullable ones).
- As of v3.25 you can use Zod v4 as a peer-dependency, so long as you still provide v3-schemas.

Looking for the exact opposite? Check out [json-schema-to-zod](#)

Sponsors

As this project will no longer be receiving updates I am no longer accepting new partnerships. Huge shout-out to CodeRabbit for keeping it real to the end <3

Cut code review time & bugs in half
coderabbit.ai

Usage

Basic example

```
import { z } from "zod"; // Or, using v3.25 or v4, "zod/v3"
import { zodToJsonSchema } from "zod-to-json-schema";

const mySchema = z
  .object({
    myString: z.string().min(5),
    myUnion: z.union([z.number(), z.boolean()]),
  })
  .describe("My neat object schema");

const jsonSchema = zodToJsonSchema(mySchema, "mySchema");
```



Expected output

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "$ref": "#/definitions/mySchema",
  "definitions": {
    "mySchema": {
      "description": "My neat object schema",
      "type": "object",
      "properties": {
        "myString": {
          "type": "string",
          "minLength": 5
        },
        "myUnion": {
          "type": ["number", "boolean"]
        }
      }
    }
  }
}
```



```
    }
  },
  "additionalProperties": false,
  "required": ["myString", "myUnion"]
}
}
```

Options

Schema name

You can pass a string as the second parameter of the main `zodToJsonSchema` function. If you do, your schema will end up inside a `definitions` object property on the root and referenced from there. Alternatively, you can pass the name as the `name` property of the options object (see below).

Options object

Instead of the schema name (or nothing), you can pass an options object as the second parameter. The following options are available:

Option	Effect
name?: <i>string</i>	As described above.
nameStrategy?: "ref" "title"	Adds name as "title" meta instead of as a ref as described above
basePath?: <i>string[]</i>	The base path of the root reference builder. Defaults to ["#"].

Option	Effect
\$refStrategy?: "root" "relative" "seen" "none"	The reference builder strategy; • "root" resolves \$refs from the root up, ie: <code>"#/definitions/mySchema"</code>. • "relative" uses relative JSON pointers. See <i>known issues!</i> • "seen" reuses the output of any "seen" Zod schema. In theory it's a more performant version of "none", but in practice this behaviour can cause issues with nested schemas and has now gotten its own option. • "none" ignores referencing all together, creating a new schema branch even on "seen" schemas. Recursive references defaults to "any", ie <code>{}</code>. Defaults to "root".
effectStrategy?: "input" "any"	The effects output strategy. Defaults to "input". See <i>known issues!</i>
dateStrategy?: "format:date" "format:date-time" "string" "integer"	Date strategy, integer allow to specify in unix-time min and max values. "format:date" creates a string schema with format: "date". "format:date-time" creates a string schema with format: "date-time". "string" is interpreted as "format:date-time". "integer" creates an integer schema with format "unix-time" (unless target "openApi3" is used min max checks are also respected)
emailStrategy?: "format:email" "format:ident-email" "pattern:zod"	Choose how to handle the email string check. Defaults to "format:email".

Option	Effect
base64Strategy?: "format:binary" "contentEncoding:base64" "pattern:zod"	Choose how to handle the base64 string check. Defaults to "contentEncoding:base64" as described here . Note that "format:binary" is not represented in the output type as it's not part of the JSON Schema spec and only intended to be used when targeting OpenAPI 3.0. Later versions of OpenAPI support contentEncoding.
definitionPath?: "definitions" "\$defs"	The name of the definitions property when name is passed. Defaults to "definitions".
target?: "jsonSchema7" "jsonSchema2019-09" "openApi3" "openAi"	Which spec to target. Defaults to "jsonSchema7"
strictUnions?: boolean	Scrubs unions of any-like json schemas, like <code>{}</code> or <code>true</code> . Multiple zod types may result in these out of necessity, such as <code>z.instanceof()</code>
definitions?: Record<string, ZodSchema>	See separate section below
errorMessages?: boolean	Include custom error messages created via chained function checks for supported zod types. See section below
markdownDescription?: boolean	Copies the <code>description</code> meta to <code>markdownDescription</code>

Option	Effect
patternStrategy?: "escape" "preserve"	The Zod string validations <code>.includes()</code> , <code>.startsWith()</code> , and <code>.endsWith()</code> must be converted to regex to be compatible with JSON Schema's <code>pattern</code> . For safety, all non-alphanumeric characters are escaped by default (consider <code>z.string().includes(".")</code>), but this can occasionally cause problems with Unicode-flagged regex parsers. Use <code>preserve</code> to prevent this escaping behaviour and preserve the exact string written, even if it results in an inaccurate regex.
applyRegexFlags?: boolean	JSON Schema's <code>pattern</code> doesn't support RegExp flags, but Zod's <code>z.string().regex()</code> does. When this option is true (default false), a best-effort is made to transform regexes into a flag-independent form (e.g. <code>/x/i => /[xX]/</code>). Supported flags: <code>i</code> (basic Latin only), <code>m</code> , <code>s</code> .
pipeStrategy?: "all" "input" "output"	Decide which types should be included when using <code>z.pipe</code> , for example <code>z.string().pipe(z.number())</code> would return both <code>string</code> and <code>number</code> by default, only <code>string</code> for "input" and only <code>number</code> for "output".
removeAdditionalStrategy?: "passthrough" "strict"	Decide when <code>additionalProperties</code> should be allowed. See the section on additional properties for details.
allowedAdditionalProperties?: true undefined	What value to give <code>additionalProperties</code> when allowed. See the section on additional properties for details.
rejectedAdditionalProperties?: false undefined	What value to give <code>additionalProperties</code> when rejected. See the section on additional properties for details.
override?: callback	See section

Option	Effect
postProcess?: callback	See section
openAiAny TypeName ?: string	Decides the name of a Json schema used to allow semi-arbitrary values in Open AI structured output. If any value in the Zod-schema resolves to any "any"-type schema it will reference a definition of this name. If no such definition is provided a default Json schema will be used. Defaults to "OpenAiAnyType"

Definitions

The definitions option lets you manually add recurring schemas into definitions for cleaner outputs. It's fully compatible with named schemas, changed definitions path and base path. Here's a simple example:

```
const myRecurringSchema = z.string();
const myObjectSchema = z.object({ a: myRecurringSchema, b:
myRecurringSchema });

const myJsonSchema = zodToJsonSchema(myObjectSchema, {
  definitions: { myRecurringSchema },
});
```

Result

```
{
  "type": "object",
  "properties": {
    "a": {
      "$ref": "#/definitions/myRecurringSchema"
    },
    "b": {
      "$ref": "#/definitions/myRecurringSchema"
    }
  },
  "definitions": {
    "myRecurringSchema": {
      "type": "string"
    }
  }
}
```

Error Messages

This feature allows optionally including error messages created via chained function calls for supported zod types:

```
// string schema with additional chained function call checks
const EmailSchema = z.string().email("Invalid email").min(5,
  "Too short");

const jsonSchema = zodToJsonSchema(EmailSchema, { errorMessages:
  true });
```

Result

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "type": "string",
  "format": "email",
  "minLength": 5,
  "errorMessage": {
    "format": "Invalid email",
    "minLength": "Too short"
  }
}
```

This allows for field specific, validation step specific error messages which can be useful for building forms and such. This format is accepted by `react-hook-form`'s `ajv` resolver (and therefore `ajv-errors` which it uses under the hood). Note that if using AJV with this format will require [enabling `ajv-errors`](#) as vanilla AJV does not accept this format by default.

Custom Error Message Support

- `ZodString`
 - `regex`
 - `min`, `max`
 - `email`, `cuid`, `uuid`, `url`
 - `endsWith`, `startsWith`
- `ZodNumber`
 - `min`, `max`, `lt`, `lte`, `gt`, `gte`,
 - `int`
 - `multipleOf`

- `ZodSet`
 - `min`, `max`
- `ZodArray`
 - `min`, `max`

Additional properties

By default, Zod removes undeclared properties when parsing object schemas. In order to replicate the expected output of this behaviour, the default for behaviour of `zodToJsonSchema` is to set `"additionalProperties"` to `false` (although the correctness of this can be debated). If you wish to allow undeclared properties you can either:

- Set `removeAdditionalStrategy` to `"strict"`. This will allow additional properties for any object schema that is not declared with `.strict()`.
- Leave `removeAdditionalStrategy` set to its default value of `"passthrough"`, and add `.passthrough()` to your object schema.

Removing the `additionalProperties` keyword using the `allowedAdditionalProperties` and/or `rejectedAdditionalProperties` options.

Some schema definitions (like Google's Gen AI API for instance) does not allow the `additionalProperties` keyword at all. Luckily the JSON Schema spec allows for this: leaving the keyword undefined *should* have the same effect as setting it to `true` (as per usual YMMV). To enable this behaviour, set the option `allowedAdditionalProperties` to `undefined`.

To exclude the keyword even when additional properties are *not* allowed, set the `rejectedAdditionalProperties` to `undefined` as well.

Heads up ⚠️: Both of these options will be ignored if your schema is declared with `.catchall(...)` as the provided schema will be used instead (if valid).

Expected outputs

<code>z.object({})</code> + option	<code>"additionalProperties"</code> value
<code>.strip()</code> (default)	<code>false</code> if strategy is <code>"passthrough"</code> , <code>true</code> if <code>"strict"</code>
<code>.passthrough()</code>	<code>true</code>
<code>.strict()</code>	<code>false</code>

<code>z.object({})</code> + option	"additionalProperties" value
<code>.catchall(z.string())</code>	<code>{ "type": "string" }</code>
<code>.catchall(z.function())</code>	<code>undefined</code> (function schemas are not currently parseable)

Substitute `true` and `false` for `undefined` according to `allowedAdditionalProperties` and/or `rejectedAdditionalProperties` respectively.

override

This options takes a callback receiving a Zod schema definition, the current reference object (containing the current ref path and other options), an argument containing information about whether or not the schema has been encountered before, and a `forceResolution` argument.

Important: if you don't want to override the current item you have to return the `ignoreOverride` symbol exported from the index. This is because `undefined` is a valid option to return when you want the property to be excluded from the resulting JSON schema.

```
import zodToJsonSchema, { ignoreOverride } from "zod-to-json-schema";
```

```
zodToJsonSchema(
  z.object({
    ignoreThis: z.string(),
    overrideThis: z.string(),
    removeThis: z.string(),
  }),
  {
    override: (def, refs) => {
      const path = refs.currentPath.join("/");

      if (path === "#/properties/overrideThis") {
        return {
          type: "integer",
        };
      }

      if (path === "#/properties/removeThis") {
        return undefined;
      }
    }
  }
)
```

```

    // Important! Do not return `undefined` or void unless you
    want to remove the property from the resulting schema
    completely.
    return ignoreOverride;
  },
},
);

```

Expected output:

```

{
  "type": "object",
  "required": ["ignoreThis", "overrideThis"],
  "properties": {
    "ignoreThis": {
      "type": "string"
    },
    "overrideThis": {
      "type": "integer"
    }
  },
  "additionalProperties": false
}

```

postProcess

Besides receiving all arguments of the `override` callback, the `postProcess` callback also receives the generated schema. It should always return a JSON Schema, or `undefined` if you wish to filter it out. Unlike the `override` callback you do not have to return `ignoreOverride` if you are happy with the produced schema; simply return it unchanged.

```

import zodToJsonSchema, { PostProcessCallback } from "zod-to-
json-schema";

// Define the callback to be used to process the output using
the PostProcessCallback type:
const postProcess: PostProcessCallback = (
  // The original output produced by the package itself:
  jsonSchema,
  // The ZodSchema def used to produce the original schema:
  def,
  // The refs object containing the current path, passed
  options, etc.
  refs,

```

```

) => {
  if (!jsonSchema) {
    return jsonSchema;
  }

  // Try to expand description as JSON meta:
  if (jsonSchema.description) {
    try {
      jsonSchema = {
        ...jsonSchema,
        ...JSON.parse(jsonSchema.description),
      };
    } catch {}
  }

  // Make all numbers nullable:
  if ("type" in jsonSchema! && jsonSchema.type === "number") {
    jsonSchema.type = ["number", "null"];
  }

  // Add the refs path, just because
  (jsonSchema as any).path = refs.currentPath;

  return jsonSchema;
};

const jsonSchema = zodToJsonSchema(zodSchema, { postProcess });

```

Using `postProcess` for including examples and other meta

Adding support for examples and other JSON Schema meta keys are among the most commonly requested features for this project. Unfortunately the current Zod major (3) has pretty anemic support for this, so some userland hacking is required. Since this is such a common usecase I've included a helper function that simply tries to parse any description as JSON and expand it into the resulting schema.

Simply stringify whatever you want added to the output schema as the description, then import and use `jsonDescription` as the `postProcess` option:

```

import zodToJsonSchema, { jsonDescription } from "zod-to-json-
schema";

const zodSchema = z.string().describe(
  JSON.stringify({
    title: "My string",
    description: "My description",
    examples: ["Foo", "Bar"],
    whatever: 123,

```



```
    }),  
  );  
  
  const jsonSchema = zodToJsonSchema(zodSchema, {  
    postProcess: jsonDescription,  
  });
```

Expected output:

```
{  
  "$schema": "http://json-schema.org/draft-07/schema#",  
  "type": "string",  
  "title": "My string",  
  "description": "My description",  
  "examples": ["Foo", "Bar"],  
  "whatever": 123  
}
```



Known issues

- The OpenAI target should be considered experimental for now, as some combination of options may break the compatibility.
- When using `.transform`, the return type is inferred from the supplied function. In other words, there is no schema for the return type, and there is no way to convert it in runtime. Currently the JSON schema will therefore reflect the input side of the Zod schema and not necessarily the output (the latter aka. `z.infer`). If this causes problems with your schema, consider using the effectStrategy "any", which will allow any type of output.
- JSON Schemas does not support any other key type than strings for objects. When using `z.record` with any other key type, this will be ignored. An exception to this rule is `z.enum` as is supported since 3.11.3
- Relative JSON pointers, while published alongside [JSON schema draft 2020-](#)

Releases 6



3.20.3

Latest

3 years ago

[+ 5 releases](#)

Sponsor this project



StefanTerdell Stefan Terdell



[Learn more about GitHub Sponsors](#)

Contributors 29



[+ 15 contributors](#)

Languages

 TypeScript 99.6%  JavaScript 0.4%